

NEO PORTFOLIO

Manual de Desarrollo

Portafolio Web Bloodpunk Cyber Demon

Autor: Diego Alanya Gamarra
Periodo académico: 2do Semestre
Turno: Diurno
Año: 2025

Documento académico y técnico
Desarrollo de una aplicación web dinámica con JSP, Java, MySQL, HTML5, CSS3, JavaScript, Apache Tomcat y Railway.

<div style="page-break-after: always;"></div>

Control del documento

Campo	Información
Proyecto	NEO PORTFOLIO
Documento	Manual de Desarrollo
Versión	1.0
Autor	Diego Alanya Gamarra
Periodo	2do Semestre
Turno	Diurno
Año	2025
Estado	Versión para presentación académica

Propósito del manual

Este manual documenta el análisis, diseño, construcción, funcionamiento y despliegue de **NEO PORTFOLIO**, un portafolio web dinámico que presenta proyectos académicos y permite ejecutar ejercicios de programación mediante formularios interactivos.

El documento se dirige a estudiantes, docentes, desarrolladores principiantes y evaluadores que necesiten comprender la arquitectura del sistema, reproducir su instalación, revisar su código o continuar su desarrollo.

Convenciones utilizadas

- Los nombres de clases Java se muestran con tipografía monoespaciada: `ProyectoDAO`.
- Los nombres de archivos se escriben con su extensión: `trabajos.jsp`.
- Los comandos se muestran en bloques de terminal.
- Las credenciales incluidas son ejemplos y deben reemplazarse en producción.
- Las rutas pueden variar según la versión de NetBeans, Maven o Apache Tomcat.

<div style="page-break-after: always;"></div>

Índice

1. Introducción
2. Alcance, objetivos y tecnologías
3. Requisitos del sistema
4. Estructura general del proyecto
5. Modelo MVC
6. Base de datos MySQL
7. Diagrama de relaciones
8. Conexión desde Java
9. Capa de acceso a datos
10. Diseño visual Bloodpunk/Cyber Demon
11. Tipografías, componentes y accesibilidad
12. Animaciones, partículas y efectos
13. Página de inicio
14. Galería de trabajos
15. Detalle de proyectos
16. Lista y resolución de ejercicios
17. Autenticación
18. Panel de administración
19. Formulario de contacto
20. Sistema de 53 ejercicios
21. Organización por semanas
22. Entradas dinámicas y cálculo en tiempo real
23. Visualización segura de código Java
24. Códigos fuente principales
25. Configuración local
26. Pruebas y validación
27. Seguridad
28. Despliegue en Railway
29. Base de datos y variables de entorno
30. Despliegue automático desde GitHub
31. Guía de uso
32. Mantenimiento y solución de problemas
33. Conclusiones y mejoras futuras

Al convertir este archivo a PDF, el índice puede transformarse en una tabla de contenido automática utilizando Pandoc, Typora, Obsidian o Visual Studio Code.

<div style="page-break-after: always;"></div>

1. Introducción

1.1 ¿Qué es NEO PORTFOLIO?

NEO PORTFOLIO es una aplicación web orientada a la presentación organizada de proyectos y ejercicios académicos de programación. A diferencia de un portafolio estático, el sistema incorpora páginas dinámicas, persistencia de datos, autenticación administrativa y calculadoras interactivas.

Su identidad visual se basa en el concepto **Bloodpunk Cyber Demon**, una combinación de estética tecnológica, fondos oscuros, acentos rojos y verdes, brillos, partículas y formas angulares. El estilo visual busca diferenciar el proyecto sin afectar la legibilidad ni la navegación.

El portafolio permite:

- Presentar información personal y académica.
- Mostrar trabajos agrupados en 14 semanas.
- Consultar el detalle de cada proyecto.
- Acceder a 53 ejercicios organizados en 11 semanas.
- Ejecutar cálculos mediante entradas dinámicas.
- Mostrar fragmentos de código Java.
- Autenticar a un usuario administrador.
- Gestionar contenidos desde un panel privado.
- Recibir mensajes mediante un formulario de contacto.

1.2 Problema identificado

Los trabajos académicos suelen almacenarse como archivos separados, capturas o carpetas locales. Esto dificulta su presentación, actualización y evaluación. NEO PORTFOLIO centraliza el contenido en una interfaz disponible desde un navegador y añade demostraciones ejecutables para evidenciar el aprendizaje.

1.3 Solución propuesta

La solución consiste en una aplicación Java web con patrón MVC. JSP se utiliza para las vistas, Java para la lógica, MySQL para la persistencia y Apache Tomcat como servidor. Railway aloja la aplicación y la base de datos en la nube.

2. Objetivos, alcance y tecnologías

2.1 Objetivo general

Desarrollar un portafolio web profesional que organice proyectos académicos, ejecute ejercicios interactivos y demuestre el uso integrado de tecnologías front-end, back-end, base de datos y despliegue en la nube.

2.2 Objetivos específicos

1. Diseñar una interfaz coherente con la identidad Bloodpunk/Cyber Demon.
2. Aplicar el patrón MVC para separar presentación, lógica y acceso a datos.
3. Registrar proyectos, ejercicios y usuarios en MySQL.
4. Implementar 53 ejercicios distribuidos en 11 semanas.
5. Crear un panel protegido para administrar el contenido.
6. Usar variables de entorno para proteger la configuración sensible.
7. Publicar la aplicación mediante un flujo automático desde GitHub.

2.3 Alcance

El proyecto cubre el registro y visualización de trabajos, la clasificación semanal de ejercicios, el cálculo interactivo, la autenticación básica, la administración de contenido y el despliegue web.

No se consideran parte de la versión inicial:

- Registro público de usuarios.
- Pagos o comercio electrónico.
- Edición colaborativa en tiempo real.
- Aplicación móvil nativa.
- Almacenamiento de contraseñas en texto plano.

2.4 Tecnologías utilizadas

Tecnología	Función
JSP	Construcción de vistas dinámicas
Java	Lógica del negocio, modelos, servlets y DAO
MySQL	Persistencia de proyectos, ejercicios y usuarios
HTML5	Estructura semántica de las páginas
CSS3	Diseño adaptable, animaciones y efectos
JavaScript	Interactividad y cálculos en tiempo real
Apache Tomcat	Contenedor de servlets

Tecnología	Función
Railway	Hosting de la aplicación y MySQL
GitHub	Control de versiones y origen del despliegue

<div style="page-break-after: always;"></div>

3. Requisitos del sistema

3.1 Requisitos funcionales

Código	Requisito
RF-01	Mostrar una página principal con presentación del autor
RF-02	Listar trabajos agrupados en 14 semanas
RF-03	Mostrar el detalle de un proyecto seleccionado
RF-04	Listar 53 ejercicios agrupados en 11 semanas
RF-05	Generar entradas apropiadas para cada ejercicio
RF-06	Calcular y mostrar resultados sin recargar cuando sea posible
RF-07	Mostrar el código Java asociado a un ejercicio
RF-08	Autenticar al administrador
RF-09	Proteger el dashboard mediante sesión
RF-10	Permitir crear, editar y eliminar contenido
RF-11	Recibir y validar mensajes de contacto
RF-12	Leer la configuración de producción desde variables de entorno

3.2 Requisitos no funcionales

- Diseño adaptable a computadoras, tabletas y teléfonos.
- Tiempo de respuesta adecuado para operaciones comunes.
- Contraseñas almacenadas mediante hash seguro.
- Consultas SQL preparadas contra inyección SQL.
- Interfaz consistente y legible.
- Código organizado en paquetes y capas.
- Compatibilidad con navegadores modernos.
- Disponibilidad mediante HTTPS en producción.

3.3 Requisitos para desarrollo local

- JDK 17 o una versión compatible con el proyecto.
- Apache Maven 3.9 o superior.
- Apache Tomcat 10.1 si se usan paquetes `jakarta.*`.
- MySQL 8.
- NetBeans, IntelliJ IDEA o Visual Studio Code.
- Git.

Si el proyecto usa `javax.servlet` , debe emplearse Tomcat 9. Si utiliza `jakarta.servlet` , corresponde Tomcat 10 o superior. No se deben mezclar ambas APIs.

<div style="page-break-after: always;"></div>

4. Estructura del proyecto

La siguiente estructura corresponde a una aplicación Maven:

```
neo-portfolio/
├─ pom.xml
├─ README.md
├─ Dockerfile
├─ src/
│   └─ main/
│       └─ java/
│           └─ pe/neoportfolio/
│               └─ config/
│                   └─ ConexionBD.java
│               └─ controller/
│                   ├── LoginServlet.java
│                   ├── LogoutServlet.java
│                   ├── ProyectoServlet.java
│                   └─ EjercicioServlet.java
│               └─ dao/
│                   ├── ProyectoDAO.java
│                   ├── EjercicioDAO.java
│                   └─ UsuarioDAO.java
│               └─ model/
│                   ├── Proyecto.java
│                   ├── Ejercicio.java
│                   └─ Usuario.java
│               └─ filter/
│                   └─ AuthFilter.java
│   └─ resources/
│       └─ database.sql
│   └─ webapp/
│       ├── assets/
│       │   ├── css/style.css
│       │   ├── js/main.js
│       │   ├── js/ejercicios.js
│       │   └─ img/
│       ├── includes/
│       │   ├── header.jspf
│       │   └─ footer.jspf
│       ├── WEB-INF/
│       │   └─ web.xml
│       ├── index.jsp
│       ├── trabajos.jsp
│       ├── ver-proyecto.jsp
│       └─ ejercicios.jsp
```

```

|   |   |   | resultado.jsp
|   |   |   | login.jsp
|   |   |   | dashboard.jsp
|   |   |   | contacto.jsp
|   |   |   |
|   |   |   | test/
|   |   |   |   | java/
|   |   |   |   |
|   |   |   |   | .gitignore

```

<div style="page-break-after: always;"></div>

5. Explicación de archivos y modelo MVC

5.1 Archivos principales

Archivo o carpeta	Responsabilidad
pom.xml	Dependencias, compilación y empaquetado WAR
ConexionBD.java	Creación de conexiones MySQL
controller/	Recepción de solicitudes HTTP
dao/	Consultas y operaciones CRUD
model/	Objetos que representan datos del sistema
filter/AuthFilter.java	Protección de rutas administrativas
database.sql	Creación inicial de tablas y datos controlados
assets/css/style.css	Sistema visual del portafolio
assets/js/ejercicios.js	Validación y cálculo interactivo
includes/*.jspf	Cabecera y pie reutilizables
WEB-INF/web.xml	Configuración web y páginas de error

5.2 Modelo MVC

NEO PORTFOLIO utiliza **Modelo–Vista–Controlador**:

```

flowchart TD
    U["Usuario"] --> V["Vista JSP"]
    V --> C["Servlet controlador"]
    C --> M["Modelo y DAO"]
    M --> DB["MySQL"]
    DB --> M
    M --> C
    C --> V

```

- **Modelo:** clases Proyecto, Ejercicio y Usuario, junto con los DAO.
- **Vista:** páginas JSP, fragmentos JSPF, HTML, CSS y JavaScript.
- **Controlador:** servlets que validan solicitudes, llaman a los DAO y seleccionan una vista.

5.3 Flujo de ejemplo

1. El usuario abre `trabajos`.
2. `ProyectoServlet` recibe la petición.
3. `ProyectoDAO` consulta la base de datos.
4. El servlet agrega la lista al objeto `request`.
5. La solicitud se reenvía a `trabajos.jsp`.
6. JSP genera las tarjetas dinámicas.

Esta separación facilita las pruebas, el mantenimiento y la reutilización.

<div style="page-break-after: always;"></div>

6. Base de datos MySQL

6.1 Creación del esquema

```
CREATE DATABASE IF NOT EXISTS neo_portfolio
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE neo_portfolio;

CREATE TABLE proyectos (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  semana TINYINT UNSIGNED NOT NULL,
  titulo VARCHAR(150) NOT NULL,
  resumen VARCHAR(300) NOT NULL,
  descripcion TEXT NOT NULL,
  imagen_url VARCHAR(500),
  repositorio_url VARCHAR(500),
  demo_url VARCHAR(500),
  tecnologias VARCHAR(300),
  publicado BOOLEAN NOT NULL DEFAULT TRUE,
  creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  actualizado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT chk_proyecto_semana CHECK (semana BETWEEN 1 AND 14)
);

CREATE INDEX idx_proyectos_semana
  ON proyectos (semana, publicado);
```

6.2 Tabla ejercicios

```
CREATE TABLE ejercicios (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  semana TINYINT UNSIGNED NOT NULL,
```



```

numero TINYINT UNSIGNED NOT NULL,
titulo VARCHAR(150) NOT NULL,
enunciado TEXT NOT NULL,
categoria VARCHAR(100) NOT NULL,
configuracion_inputs JSON NOT NULL,
codigo_java MEDIUMTEXT,
funcion_js VARCHAR(80) NOT NULL,
activo BOOLEAN NOT NULL DEFAULT TRUE,
creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT chk_ejercicio_semana CHECK (semana BETWEEN 1 AND 11),
CONSTRAINT uq_ejercicio_semana_numero UNIQUE (semana, numero)
);

CREATE INDEX idx_ejercicios_semana
ON ejercicios (semana, activo);

```

La columna `configuracion_inputs` describe los controles que requiere cada ejercicio sin duplicar páginas JSP.

<div style="page-break-after: always;"></div>

7. Usuarios y diagrama de relaciones

7.1 Tabla `usuarios`

```

CREATE TABLE usuarios (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL,
    username VARCHAR(60) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    rol ENUM('ADMIN', 'EDITOR') NOT NULL DEFAULT 'EDITOR',
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    ultimo_acceso TIMESTAMP NULL,
    creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

No se almacena una contraseña real en `password_hash`. El valor debe generarse mediante BCrypt o Argon2.

7.2 Relación lógica de administración

Para conservar el historial de cambios se pueden añadir `creado_por` y `actualizado_por` en proyectos y ejercicios. La versión mínima funciona sin esas columnas; la versión ampliada puede establecer claves foráneas hacia `usuarios.id`.

```

erDiagram
    USUARIOS ||--o{ PROYECTOS : administra
    USUARIOS ||--o{ EJERCICIOS : administra

    USUARIOS {
        bigint id PK
        varchar username UK
        varchar password_hash
        enum rol
        boolean activo
    }

    PROYECTOS {
        bigint id PK
        tinyint semana
        varchar titulo
        text descripcion
        boolean publicado
        bigint actualizado_por FK
    }

    EJERCICIOS {
        bigint id PK
        tinyint semana
        tinyint numero
        text enunciado
        json configuracion_inputs
        bigint actualizado_por FK
    }

```

7.3 Reglas de integridad

- Una semana de proyecto debe estar entre 1 y 14.
- Una semana de ejercicios debe estar entre 1 y 11.
- No puede repetirse el número de ejercicio dentro de una semana.
- El nombre de usuario debe ser único.
- Los registros pueden desactivarse sin eliminarlos físicamente.

<div style="page-break-after: always;"></div>

8. Conexión desde Java: ConexionBD.java

```

package pe.neoportfolio.config;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

```

```

public final class ConexionBD {

    private static final String URL = leer(
        "MYSQL_URL",
        "jdbc:mysql://localhost:3306/neo_portfolio"
        + "?useSSL=false&serverTimezone=UTC"
    );

    private static final String USER = leer("MYSQLUSER", "root");
    private static final String PASSWORD = leer("MYSQLPASSWORD", "");

    private ConexionBD() {
        // Evita crear instancias de esta clase de utilidad.
    }

    public static Connection obtenerConexion() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    private static String leer(String nombre, String valorLocal) {
        String valor = System.getenv(nombre);
        return valor == null || valor.isBlank() ? valorLocal : valor;
    }
}

```

8.1 Consideraciones

- En producción, `MYSQL_URL` debe contener una URL JDBC.
- No se deben escribir credenciales reales dentro del repositorio.
- Cada conexión debe cerrarse con `try-with-resources`.
- Para mayor carga se recomienda un pool como HikariCP.
- Los errores internos se registran en el servidor, pero no se muestran completos al usuario.

8.2 Dependencia del conector MySQL

```

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
</dependency>

```

La versión debe verificarse y mantenerse actualizada en el proyecto real.

<div style="page-break-after: always;"></div>

9. Capa de acceso a datos

El patrón DAO concentra las consultas SQL y evita colocar acceso a datos dentro de JSP.

9.1 Operaciones esperadas

ProyectoDAO

- `listarPublicados()`
- `buscarPorId(long id)`
- `listarPorSemana(int semana)`
- `insertar(Proyecto proyecto)`
- `actualizar(Proyecto proyecto)`
- `cambiarEstado(long id, boolean publicado)`

EjercicioDAO

- `listarPorSemana(int semana)`
- `buscarPorId(long id)`
- `insertar(Ejercicio ejercicio)`
- `actualizar(Ejercicio ejercicio)`

UsuarioDAO

- `buscarPorUsername(String username)`
- `actualizarUltimoAcceso(long id)`
- `cambiarPassword(long id, String hash)`

9.2 Consulta preparada

```
String sql = ""
    SELECT id, semana, titulo, resumen, descripcion,
           imagen_url, repositorio_url, demo_url,
           tecnologias, publicado
    FROM proyectos
    WHERE publicado = TRUE
    ORDER BY semana DESC, id DESC
"";

try (Connection cn = ConexionBD.obtenerConexion();
     PreparedStatement ps = cn.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        // Convertir cada fila en un objeto Proyecto.
    }
}
```

Las consultas preparadas separan el código SQL de los valores proporcionados por el usuario y reducen el riesgo de inyección SQL.

<div style="page-break-after: always;"></div>

10. Diseño visual Bloodpunk/Cyber Demon

10.1 Concepto

El sistema de diseño combina tecnología futurista con una estética oscura y enérgica. El término “Bloodpunk” se expresa mediante acentos rojos, cortes geométricos y contrastes intensos; “Cyber Demon” se representa con brillo verde, líneas técnicas y elementos digitales. Se evita el contenido gráfico: la identidad se construye exclusivamente con color, forma, tipografía y movimiento.

10.2 Paleta

Token	Color	Uso
--color-bg	#0a0a0a	Fondo principal
--color-panel	#121212	Tarjetas y paneles
--color-red	#dc2626	Acciones y títulos
--color-green	#22c55e	Estado, éxito y detalles
--color-text	#f5f5f5	Texto principal
--color-muted	#a3a3a3	Texto secundario
--color-border	#2a2a2a	Bordes discretos

```
:root {
  --color-bg: #0a0a0a;
  --color-panel: #121212;
  --color-red: #dc2626;
  --color-green: #22c55e;
  --color-text: #f5f5f5;
  --color-muted: #a3a3a3;
  --color-border: #2a2a2a;
  --shadow-red: 0 0 24px rgb(220 38 38 / 0.28);
  --shadow-green: 0 0 18px rgb(34 197 94 / 0.22);
  --radius: 14px;
}
```

10.3 Criterio de contraste

El rojo y el verde se usan como acentos, no como color de párrafos extensos. Los mensajes importantes incluyen texto o iconos además del color para favorecer la accesibilidad.

<div style="page-break-after: always;"></div>

11. Tipografías, componentes y accesibilidad

11.1 Tipografías

- **Orbitron:** títulos principales, logotipo y cifras destacadas.
- **Rajdhani:** subtítulos, navegación y etiquetas.
- **Inter:** párrafos, formularios y textos largos.

```
body {  
  margin: 0;  
  color: var(--color-text);  
  background: var(--color-bg);  
  font-family: "Inter", system-ui, sans-serif;  
  line-height: 1.6;  
}  
  
h1, h2 {  
  font-family: "Orbitron", sans-serif;  
  letter-spacing: 0.04em;  
}  
  
nav, .eyebrow, .card__meta {  
  font-family: "Rajdhani", sans-serif;  
  font-weight: 600;  
}
```

11.2 Componentes

- Barra superior fija.
- Botón primario rojo.
- Botón secundario con borde verde.
- Tarjetas de proyecto.
- Chips de tecnología.
- Campos de formulario.
- Bloques de código.
- Alertas de éxito y error.
- Modal de confirmación.

11.3 Accesibilidad

- Usar elementos semánticos: `header`, `nav`, `main`, `section` y `footer`.
- Asociar cada `label` con su `input`.
- Mantener navegación completa por teclado.
- Mostrar un foco visible.

- Añadir texto alternativo a imágenes informativas.
- No iniciar animaciones intensas cuando el usuario prefiere movimiento reducido.

```
:focus-visible {  
  outline: 3px solid var(--color-green);  
  outline-offset: 3px;  
}  
  
@media (prefers-reduced-motion: reduce) {  
  *, *::before, *::after {  
    scroll-behavior: auto !important;  
    animation-duration: 0.01ms !important;  
    animation-iteration-count: 1 !important;  
  }  
}
```

<div style="page-break-after: always;"></div>

12. Animaciones, partículas y efectos visuales

12.1 Partículas flotantes

Las partículas deben colocarse en una capa decorativa sin bloquear botones ni enlaces.

```
.particles {  
  position: fixed;  
  inset: 0;  
  overflow: hidden;  
  pointer-events: none;  
  z-index: -1;  
}  
  
.particle {  
  position: absolute;  
  width: 4px;  
  height: 4px;  
  border-radius: 50%;  
  background: var(--color-red);  
  box-shadow: var(--shadow-red);  
  animation: float 9s linear infinite;  
}  
  
@keyframes float {  
  from { transform: translateY(110vh) scale(0.7); opacity: 0; }  
  20% { opacity: 0.7; }  
  to { transform: translateY(-15vh) scale(1.2); opacity: 0; }  
}
```

12.2 Hover y glow

```
.project-card {
  border: 1px solid var(--color-border);
  background: linear-gradient(145deg, #151515, #0d0d0d);
  border-radius: var(--radius);
  transition: transform 180ms ease, border-color 180ms ease,
              box-shadow 180ms ease;
}

.project-card:hover {
  transform: translateY(-6px);
  border-color: var(--color-red);
  box-shadow: var(--shadow-red);
}
```

12.3 Uso responsable de animaciones

Las animaciones deben reforzar la jerarquía visual. Una página con demasiados destellos reduce la legibilidad y puede disminuir el rendimiento. Se recomienda animar solo transformaciones y opacidad, pues suelen ser más eficientes que modificar dimensiones.

<div style="page-break-after: always;"></div>

13. index.jsp: página de inicio

La página principal contiene una sección hero, presentación, tecnologías, trabajos destacados y llamada a la acción.

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ include file="/includes/header.jspf" %>

<main>
  <section class="hero" aria-labelledby="hero-title">
    <div class="hero__content">
      <p class="eyebrow">DISEÑO Y PROGRAMACIÓN WEB</p>
      <h1 id="hero-title">
        NEO <span>PORTFOLIO</span>
      </h1>
      <p>
        Proyectos, ejercicios y experiencias de desarrollo
        construidas con Java y tecnologías web.
      </p>
      <div class="hero__actions">
        <a class="btn btn--primary"
          href="{pageContext.request.contextPath}/trabajos">
          Ver trabajos
```



```

</a>
<a class="btn btn--outline"
    href="{pageContext.request.contextPath}/contacto.jsp">
    Contacto
</a>
</div>
</div>
</section>

<section class="featured" aria-labelledby="featured-title">
    <h2 id="featured-title">Proyectos destacados</h2>
    <!-- Las tarjetas pueden provenir de ProyectoServlet. -->
</section>
</main>

<%@ include file="/includes/footer.jspf" %>

```

Recomendaciones

- Un solo `h1` por página.
- Botones con nombres claros.
- Imagen principal optimizada en WebP o AVIF.
- Contenido visible aun si JavaScript no carga.

```
<div style="page-break-after: always;"></div>
```

14. trabajos.jsp: galería de 14 semanas

La vista recibe `proyectos` desde el controlador y genera una tarjeta por registro.

```

<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ include file="/includes/header.jspf" %>

<main class="container">
    <header class="page-header">
        <p class="eyebrow">14 SEMANAS</p>
        <h1>Trabajos desarrollados</h1>
    </header>

    <div class="project-grid">
        <c:forEach var="proyecto" items="{proyectos}">
            <article class="project-card">
                "
                    loading="lazy">

                <div class="project-card__body">

```

```

        <span class="week-badge">
            Semana <c:out value="\${proyecto.semana}"/>
        </span>
        <h2><c:out value="\${proyecto.titulo}"/></h2>
        <p><c:out value="\${proyecto.resumen}"/></p>
        <a class="btn btn--small"
            href="\${pageContext.request.contextPath}/proyecto?id=\${proyecto.id}">
            Ver proyecto
        </a>
    </div>
</article>
</c:forEach>
</div>
</main>

<%@ include file="/includes/footer.jspf" %>

```

`<c:out>` escapa el texto y reduce el riesgo de insertar HTML no deseado.

`<div style="page-break-after: always;"></div>`

15. ver-proyecto.jsp: detalle del proyecto

La página presenta información ampliada del trabajo seleccionado.

Contenido recomendado

- Número de semana.
- Título y resumen.
- Imagen o captura principal.
- Problema resuelto.
- Objetivos.
- Tecnologías utilizadas.
- Proceso de desarrollo.
- Resultados obtenidos.
- Enlace a demostración y repositorio, si son públicos.

```

<main class="container project-detail">
    <c:choose>
        <c:when test="\${not empty proyecto}">
            <a class="back-link"
                href="\${pageContext.request.contextPath}/trabajos">
                ← Volver a trabajos
            </a>

            <span class="week-badge">
                Semana <c:out value="\${proyecto.semana}"/>
            </span>

```

```

</span>
<h1><c:out value="\${proyecto.titulo}"/></h1>
<p class="lead"><c:out value="\${proyecto.resumen}"/></p>



<section>
  <h2>Descripción</h2>
  <p><c:out value="\${proyecto.descripcion}"/></p>
</section>
</c:when>
<c:otherwise>
  <h1>Proyecto no encontrado</h1>
  <p>Verifique el enlace o regrese a la galería.</p>
</c:otherwise>
</c:choose>
</main>

```

El servlet debe validar que `id` sea numérico y responder con estado 404 cuando no exista el proyecto.

```
<div style="page-break-after: always;"></div>
```

16. ejercicios.jsp y resultado.jsp

16.1 Lista de ejercicios

`ejercicios.jsp` muestra filtros por semana y tarjetas con categoría, número y descripción.

```

<nav class="week-filter" aria-label="Semanas de ejercicios">
  <c:forEach begin="1" end="11" var="semana">
    <a href="\${pageContext.request.contextPath}/ejercicios?semana=\${semana}">
      Semana \${semana}
    </a>
  </c:forEach>
</nav>

```

16.2 Resultado interactivo

`resultado.jsp` recibe un ejercicio y genera el formulario según `configuracion_inputs`. Para un proyecto educativo pequeño, los esquemas pueden mapearse en Java antes de llegar a JSP. En una versión más avanzada se puede utilizar una biblioteca JSON.

```

<form id="exercise-form" data-operation="areaRectangulo">
  <label for="base">Base</label>
  <input id="base" name="base" type="number" step="any" required>

```

```

<label for="altura">Altura</label>
<input id="altura" name="altura" type="number" step="any" required>

<button type="submit" class="btn btn--primary">Calcular</button>
</form>

<output id="exercise-result" aria-live="polite">
  Ingrese los datos para calcular.
</output>

```

El atributo `aria-live="polite"` permite que tecnologías de asistencia anuncien el resultado actualizado.

<div style="page-break-after: always;"></div>

17. login.jsp: autenticación

17.1 Flujo

```

flowchart TD
    A["Formulario de acceso"] --> B["LoginServlet"]
    B --> C{"Datos válidos"}
    C -- "No" --> D["Mensaje genérico"]
    C -- "Sí" --> E["Crear sesión"]
    E --> F["Dashboard"]

```

17.2 Formulario

```

<form action="\${pageContext.request.contextPath}/login"
  method="post" class="auth-card">
  <h1>Acceso administrativo</h1>

  <label for="username">Usuario</label>
  <input id="username" name="username"
    autocomplete="username" required>

  <label for="password">Contraseña</label>
  <input id="password" name="password" type="password"
    autocomplete="current-password" required>

  <button class="btn btn--primary" type="submit">
    Iniciar sesión
  </button>

  <c:if test="\${not empty error}">
    <p class="alert alert--error" role="alert">
      <c:out value="\${error}"/>
    </p>
  </c:if>

```

```
</p>
</c:if>
</form>
```

17.3 Reglas

- Comparar hashes, nunca contraseñas en texto plano.
- Cambiar el identificador de sesión después del acceso.
- Cerrar la sesión desde una acción POST.
- Mostrar “Usuario o contraseña incorrectos” sin revelar cuál campo falló.
- Bloquear o limitar intentos repetidos en una versión de producción.

<div style="page-break-after: always;"></div>

18. dashboard.jsp: administración

El dashboard reúne indicadores y operaciones CRUD.

18.1 Secciones

- Resumen: cantidad de proyectos, ejercicios y semanas.
- Proyectos: crear, editar, publicar o archivar.
- Ejercicios: modificar enunciados, entradas y código.
- Cuenta: cambio de contraseña y cierre de sesión.

```
<main class="dashboard">
  <aside class="dashboard__sidebar">
    <h1>NEO ADMIN</h1>
    <nav>
      <a href="#resumen">Resumen</a>
      <a href="#proyectos">Proyectos</a>
      <a href="#ejercicios">Ejercicios</a>
    </nav>
  </aside>

  <section class="dashboard__content">
    <h2>Bienvenido,
      <c:out value="${sessionScope.usuario.nombre}"/>
    </h2>

    <div class="stats-grid">
      <article class="stat-card">
        <span>Proyectos</span>
        <strong><c:out value="${totalProyectos}"/></strong>
      </article>
      <article class="stat-card">
```

```
<span>Ejercicios</span>
<strong><c:out value="\${totalEjercicios}"/></strong>
</article>
</div>
</section>
</main>
```

18.2 Protección

El filtro `AuthFilter` comprueba que exista una sesión autenticada antes de permitir rutas como `/admin/*`. La interfaz oculta enlaces administrativos al visitante, pero la seguridad real se aplica en el servidor.

<div style="page-break-after: always;"></div>

19. contacto.jsp: formulario de contacto

19.1 Campos

- Nombre.
- Correo electrónico.
- Asunto.
- Mensaje.
- Confirmación de envío.

```
<form action="\${pageContext.request.contextPath}/contacto"
      method="post" class="contact-form">
  <label for="nombre">Nombre</label>
  <input id="nombre" name="nombre" maxlength="120" required>

  <label for="correo">Correo</label>
  <input id="correo" name="correo" type="email"
        maxlength="180" required>

  <label for="asunto">Asunto</label>
  <input id="asunto" name="asunto" maxlength="160" required>

  <label for="mensaje">Mensaje</label>
  <textarea id="mensaje" name="mensaje"
           rows="7" maxlength="2000" required></textarea>

  <button type="submit" class="btn btn--primary">
    Enviar mensaje
  </button>
</form>
```

19.2 Validación

La validación del navegador mejora la experiencia, pero el servlet debe volver a validar:

- Longitud de cada campo.
- Formato del correo.
- Campos vacíos.
- Caracteres de control.
- Frecuencia de envíos.

En producción debe añadirse protección anti-spam y un mecanismo de correo o persistencia. Nunca se deben imprimir datos del formulario directamente sin escaparlos.

```
<div style="page-break-after: always;"></div>
```

20. Sistema de 53 ejercicios

20.1 Propósito

El sistema demuestra conceptos de programación mediante problemas ejecutables. Cada ejercicio posee:

- Identificador.
- Semana y número.
- Título.
- Enunciado.
- Categoría.
- Definición de entradas.
- Función de cálculo.
- Código Java de referencia.

20.2 Distribución

Semana	Cantidad	Tema principal
1	10	Geometría y aritmética
2	6	Descuentos y sueldos
3	6	Descuentos y transporte
4	4	Alquiler, horas extras y estacionamiento
5	6	Series, dígitos y factoriales
6	3	Ventas y descuentos
7	10	Manipulación de cadenas
8	2	Math.PI y Math.hypot
9	2	Dados RPG y ecuaciones
10	2	Trigonometría y anti-desborde

Semana	Cantidad	Tema principal
11	2	Interés compuesto y distancia
Total	53	

20.3 Arquitectura reutilizable

No se crean 53 páginas diferentes. Una sola vista interpreta la configuración de cada ejercicio, genera sus controles y selecciona la función correspondiente. Esto reduce duplicación y simplifica el mantenimiento.

21. Catálogo de ejercicios: semanas 1 a 4

Los títulos siguientes funcionan como catálogo de referencia y pueden adaptarse a los enunciados originales del curso.

Semana 1 — 10 ejercicios

1. Área y perímetro de un rectángulo.
2. Área de un triángulo.
3. Área y longitud de una circunferencia.
4. Conversión de grados Celsius a Fahrenheit.
5. Promedio de tres notas.
6. Suma, resta, producto y división.
7. Conversión de segundos a horas, minutos y segundos.
8. Cálculo del IGV y total de una compra.
9. Distancia entre dos puntos.
10. Intercambio de dos valores.

Semana 2 — 6 ejercicios

1. Descuento por monto de compra.
2. Sueldo con bonificación.
3. Comisión por ventas.
4. Pago semanal por horas trabajadas.
5. Precio final con impuesto y descuento.
6. Reparto de presupuesto por porcentajes.

Semana 3 — 6 ejercicios

1. Descuento según categoría.

2. Costo de pasaje por distancia.
3. Tarifa de transporte por zona.
4. Compra de entradas con promoción.
5. Envío según peso del paquete.
6. Conversión y comparación de monedas.

Semana 4 — 4 ejercicios

1. Alquiler de vehículo por días.
2. Sueldo con horas extras.
3. Tarifa de estacionamiento por horas.
4. Pago de llamada según duración y zona.

Cada cálculo debe validar rangos, evitar divisiones por cero y mostrar unidades en el resultado.

<div style="page-break-after: always;"></div>

22. Catálogo de ejercicios: semanas 5 a 11

Semana 5 — 6 ejercicios

1. Generación de una serie aritmética.
2. Suma de los primeros números naturales.
3. Conteo de dígitos.
4. Inversión de dígitos.
5. Cálculo de factorial.
6. Verificación de número primo.

Semana 6 — 3 ejercicios

1. Gestión de ventas y ofertas.
2. Descuento acumulado por cantidad.
3. Obsequio según unidades compradas.

Semana 7 — 10 ejercicios

1. Longitud de una cadena.
2. Conversión a mayúsculas y minúsculas.
3. Extracción de iniciales.
4. Conteo de vocales.
5. Inversión de texto.

6. Verificación de palíndromo.
7. Sustitución de palabras.
8. Enmascaramiento de tarjeta.
9. Separación de nombres y apellidos.
10. Frecuencia de un carácter.

Semanas 8 a 11 — 8 ejercicios

- Semana 8: área circular con `Math.PI` ; distancia con `Math.hypot` .
- Semana 9: simulador matemático de dados RPG; solución de ecuación cuadrática.
- Semana 10: funciones trigonométricas; potencias con control de desborde.
- Semana 11: interés compuesto; distancia recorrida o euclidiana.

<div style="page-break-after: always;"></div>

23. Entradas dinámicas y cálculo en tiempo real

23.1 Configuración de ejemplo

```
[
  {
    "name": "base",
    "label": "Base",
    "type": "number",
    "step": "any",
    "min": "0",
    "required": true
  },
  {
    "name": "altura",
    "label": "Altura",
    "type": "number",
    "step": "any",
    "min": "0",
    "required": true
  }
]
```

23.2 Registro de funciones

```
const operations = {
  areaRectangulo(values) {
    const base = Number(values.base);
    const altura = Number(values.altura);
    requirePositive(base, "La base");
```

```

    requirePositive(altura, "La altura");
    return `Área: ${ (base * altura).toFixed(2) } u²`;
  },

  factorial(values) {
    const n = Number(values.numero);
    if (!Number.isInteger(n) || n < 0 || n > 170) {
      throw new Error("Ingresa un entero entre 0 y 170.");
    }
    let result = 1;
    for (let i = 2; i <= n; i += 1) result *= i;
    return `${n}! = ${result}`;
  }
};

function requirePositive(value, label) {
  if (!Number.isFinite(value) || value <= 0) {
    throw new Error(`${label} debe ser mayor que cero.`);
  }
}

```

23.3 Evento de cálculo

```

document.querySelector("#exercise-form")
  ?.addEventListener("input", calculate);

function calculate(event) {
  const form = event.currentTarget;
  const output = document.querySelector("#exercise-result");
  const values = Object.fromEntries(new FormData(form));
  const operation = operations[form.dataset.operation];

  try {
    output.textContent = operation(values);
    output.className = "result result--success";
  } catch (error) {
    output.textContent = error.message;
    output.className = "result result--error";
  }
}

```

<div style="page-break-after: always;"></div>

24. Visualización de código Java

El código almacenado en la base de datos debe mostrarse como texto, no interpretarse como HTML.

```
<section class="code-panel" aria-labelledby="code-title">
  <div class="code-panel__header">
    <h2 id="code-title">Código Java</h2>
    <button type="button" id="copy-code">Copiar</button>
  </div>

  <pre><code id="java-code"><c:out
    value="\${ejercicio.codigoJava}"/></code></pre>
</section>
```

```
document.querySelector("#copy-code")
  ?.addEventListener("click", async () => {
    const code = document.querySelector("#java-code").textContent;
    await navigator.clipboard.writeText(code);
  });
```

Ejemplo mostrado

```
public static double areaRectangulo(double base, double altura) {
    if (base <= 0 || altura <= 0) {
        throw new IllegalArgumentException(
            "Las medidas deben ser mayores que cero"
        );
    }
    return base * altura;
}
```

En producción se puede integrar resaltado de sintaxis. Si se usa una biblioteca externa, debe cargarse desde una fuente confiable, fijar su versión y revisar su licencia.

<div style="page-break-after: always;"></div>

25. Código principal: ProyectoDAO.java

```
package pe.neoportfolio.dao;

import pe.neoportfolio.config.ConexionBD;
import pe.neoportfolio.model.Proyecto;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class ProyectoDAO {

    public List<Proyecto> listarPublicados() throws SQLException {
```

```
String sql = ""
        SELECT id, semana, titulo, resumen, descripcion,
               imagen_url, repositorio_url, demo_url,
               tecnologias, publicado
        FROM proyectos
        WHERE publicado = TRUE
        ORDER BY semana DESC, id DESC
        """;
```

```
List<Proyecto> proyectos = new ArrayList<>();
```

```
try (Connection cn = ConexionBD.obtenerConexion();
     PreparedStatement ps = cn.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        proyectos.add(mapear(rs));
    }
}
return proyectos;
}
```

```
public Optional<Proyecto> buscarPorId(long id) throws SQLException {
    String sql = ""
        SELECT id, semana, titulo, resumen, descripcion,
               imagen_url, repositorio_url, demo_url,
               tecnologias, publicado
        FROM proyectos
        WHERE id = ? AND publicado = TRUE
        """;
```

```
try (Connection cn = ConexionBD.obtenerConexion();
     PreparedStatement ps = cn.prepareStatement(sql)) {
    ps.setLong(1, id);
    try (ResultSet rs = ps.executeQuery()) {
        return rs.next()
            ? Optional.of(mapear(rs))
            : Optional.empty();
    }
}
}
```

```
private Proyecto mapear(ResultSet rs) throws SQLException {
    Proyecto p = new Proyecto();
    p.setId(rs.getLong("id"));
    p.setSemana(rs.getInt("semana"));
    p.setTitulo(rs.getString("titulo"));
    p.setResumen(rs.getString("resumen"));
    p.setDescripcion(rs.getString("descripcion"));
    p.setImagenUrl(rs.getString("imagen_url"));
    p.setRepositorioUrl(rs.getString("repositorio_url"));
    p.setDemoUrl(rs.getString("demo_url"));
    p.setTecnologias(rs.getString("tecnologias"));
}
```

```

        p.setPublicado(rs.getBoolean("publicado"));
        return p;
    }
}

```

<div style="page-break-after: always;"></div>

26. Código principal: UsuarioDAO.java

```

package pe.neoportfolio.dao;

import pe.neoportfolio.config.ConexionBD;
import pe.neoportfolio.model.Usuario;

import java.sql.*;
import java.util.Optional;

public class UsuarioDAO {

    public Optional<Usuario> buscarPorUsername(String username)
        throws SQLException {
        String sql = ""
            SELECT id, nombre, username, password_hash, rol, activo
            FROM usuarios
            WHERE username = ? AND activo = TRUE
            LIMIT 1
            "";

        try (Connection cn = ConexionBD.obtenerConexion();
            PreparedStatement ps = cn.prepareStatement(sql)) {
            ps.setString(1, username);

            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return Optional.empty();

                Usuario usuario = new Usuario();
                usuario.setId(rs.getLong("id"));
                usuario.setNombre(rs.getString("nombre"));
                usuario.setUsername(rs.getString("username"));
                usuario.setPasswordHash(rs.getString("password_hash"));
                usuario.setRol(rs.getString("rol"));
                usuario.setActivo(rs.getBoolean("activo"));
                return Optional.of(usuario);
            }
        }

        public void actualizarUltimoAcceso(long id) throws SQLException {
            String sql = ""
                UPDATE usuarios
                SET ultimo_acceso = CURRENT_TIMESTAMP

```

```

        WHERE id = ?
        """;

        try (Connection cn = ConexionBD.obtenerConexion();
            PreparedStatement ps = cn.prepareStatement(sql)) {
            ps.setLong(1, id);
            ps.executeUpdate();
        }
    }
}

```

Verificación en `LoginServlet`

```

boolean passwordValido =
    BCrypt.checkpw(passwordIngresado, usuario.getPasswordHash());

```

La dependencia BCrypt puede implementarse con una biblioteca mantenida. La versión exacta debe revisarse antes de compilar.

<div style="page-break-after: always;"></div>

27. Configuración local y ejecución

27.1 Preparar la base de datos

1. Iniciar MySQL.
2. Crear el esquema ejecutando `database.sql`.
3. Configurar un usuario de desarrollo con permisos limitados.
4. Registrar un administrador con una contraseña convertida a hash.

27.2 Variables locales

En el IDE o servidor:

```

MYSQL_URL=jdbc:mysql://localhost:3306/neo_portfolio?useSSL=false&serverTimezone=UTC
MYSQLUSER=neo_app
MYSQLPASSWORD=CAMBIAR_SOLO_EN_EL_ENTORNO_LOCAL

```

No se debe subir un archivo con secretos al repositorio. Puede conservarse un archivo `.env.example` sin valores reales:

```

MYSQL_URL=
MYSQLUSER=

```

```
MYSQLPASSWORD=  
SESSION_SECRET=
```

27.3 Compilar

```
mvn clean test  
mvn clean package
```

El archivo generado normalmente se encuentra en:

```
target/neo-portfolio.war
```

27.4 Ejecutar en Tomcat

Copiar el WAR al directorio `webapps` de Tomcat o configurar el servidor desde NetBeans. Después, acceder a una ruta similar:

```
http://localhost:8080/neo-portfolio/
```

<div style="page-break-after: always;"></div>

28. Pruebas y validación

28.1 Estrategia

Tipo	Ejemplo
Unitaria	Verificar área, factorial e interés compuesto
Integración	Consultar proyectos desde MySQL
Funcional	Iniciar sesión y abrir el dashboard
Interfaz	Revisar responsive y navegación por teclado
Seguridad	Probar consultas maliciosas y sesión expirada
Despliegue	Verificar variables y conexión en Railway

28.2 Casos esenciales

Caso	Entrada	Resultado esperado
Área válida	base 5, altura 4	20 u ²
Área inválida	base -1	Mensaje de validación
Factorial válido	5	120
División	divisor 0	Operación rechazada
Login correcto	datos válidos	Dashboard
Login incorrecto	contraseña inválida	Mensaje genérico
Proyecto inexistente	id desconocido	Página 404

Caso	Entrada	Resultado esperado
Ruta protegida	sin sesión	Redirección al login

28.3 Revisión visual

Probar como mínimo:

- 360 px de ancho.
- 768 px de ancho.
- 1366 px de ancho.
- Navegación con teclado.
- Preferencia de movimiento reducido.
- Zoom del navegador al 200 %.

28.4 Criterio de aceptación

Una versión es apta para entrega cuando compila sin errores, las páginas principales responden correctamente, los 53 ejercicios tienen entradas y resultados, el dashboard está protegido y el despliegue público no revela secretos.

<div style="page-break-after: always;"></div>

29. Seguridad de la aplicación

29.1 Controles mínimos

- Hash de contraseñas con BCrypt o Argon2.
- Consultas SQL preparadas.
- Escape de salida con JSTL.
- Validación en servidor.
- Cookies de sesión `HttpOnly`, `Secure` y `SameSite`.
- Invalidación de sesión durante el cierre.
- Protección CSRF para operaciones de cambio.
- Límites de tamaño en formularios.
- Mensajes de error genéricos.
- HTTPS en producción.

29.2 Credenciales

El manual no debe contener credenciales reales. Para una demostración académica puede documentarse:

```
Usuario de demostración: admin_demo
Contraseña: definida por el responsable durante la instalación
```

En Railway, la contraseña se configura como variable privada. En una exposición, el docente puede recibirla por un medio separado si necesita probar la administración.

29.3 Encabezados recomendados

```
Content-Security-Policy
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy
```

Una política CSP debe adaptarse a los recursos reales del proyecto. No conviene copiar una política restrictiva sin probarla, pues podría bloquear fuentes o scripts necesarios.

<div style="page-break-after: always;"></div>

30. Despliegue en Railway

30.1 Arquitectura de producción

```
flowchart TD
    G["Repositorio GitHub"] --> R["Servicio Railway"]
    R --> T["Aplicación Java/Tomcat"]
    T --> M["MySQL Railway"]
    U["Usuario web"] --> T
```

30.2 Flujo general

1. Subir el código a un repositorio GitHub.
2. Crear un proyecto en Railway.
3. Añadir un servicio desde el repositorio.
4. Añadir un servicio MySQL.
5. Configurar variables de entorno.
6. Definir el comando de compilación y ejecución.
7. Esperar la construcción.
8. revisar los registros.
9. Generar o asociar el dominio público.

30.3 Dominio documentado

```
https://neoportfolio-production.up.railway.app
```

La disponibilidad exacta del dominio depende de que el servicio se encuentre activo y de que el nombre haya sido asignado en Railway.

30.4 Dockerfile de referencia

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM tomcat:10.1-jdk17-temurin
RUN rm -rf /usr/local/tomcat/webapps/*
COPY --from=build /app/target/*.war \
    /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

<div style="page-break-after: always;"></div>

31. MySQL y variables de entorno en Railway

31.1 Variables esperadas

Variable	Propósito
MYSQL_URL	URL JDBC completa
MYSQLUSER	Usuario de MySQL
MYSQLPASSWORD	Contraseña de MySQL
MYSQLHOST	Host interno
MYSQLPORT	Puerto
MYSQLDATABASE	Nombre de la base
SESSION_SECRET	Secreto adicional de la aplicación

Railway puede proporcionar variables con nombres específicos. La aplicación debe mapear esos nombres sin copiar los valores al código.

31.2 Construcción de URL alternativa

Si no existe una URL JDBC completa:

```
String host = System.getenv("MYSQLHOST");
String port = System.getenv("MYSQLPORT");
String database = System.getenv("MYSQLDATABASE");
```

```
String url = "jdbc:mysql://" + host + ":" + port + "/" + database
+ "?useSSL=true&serverTimezone=UTC";
```

31.3 Migración inicial

El archivo `database.sql` puede ejecutarse una vez desde un cliente autorizado. Para proyectos mayores se recomienda Flyway o Liquibase, que permiten versionar cambios de estructura.

31.4 Verificación

- Confirmar que la aplicación inicia.
- Revisar que las tablas existan.
- Crear un registro de prueba.
- Reiniciar el servicio y comprobar persistencia.
- Verificar que los registros no impriman contraseñas.

<div style="page-break-after: always;"></div>

32. Deploy automático desde GitHub

32.1 Flujo de versiones

```
flowchart LR
    A["Cambio local"] --> B["Commit"]
    B --> C["Push a GitHub"]
    C --> D["Build automático"]
    D --> E{"Pruebas y build"}
    E -- "Correcto" --> F["Nueva versión"]
    E -- "Error" --> G["Revisar logs"]
```

32.2 Buenas prácticas de Git

```
git checkout -b feature/galeria-proyectos
git add .
git commit -m "feat: agrega galería dinámica de trabajos"
git push -u origin feature/galeria-proyectos
```

Antes de integrar a la rama de producción:

- Ejecutar pruebas.
- Revisar que no existan secretos.
- Confirmar que `.gitignore` excluya archivos locales.

- Revisar la compatibilidad con el JDK configurado.
- Mantener mensajes de commit claros.

32.3 Archivos que no deben subirse

```
target/  
.idea/  
.vscode/  
nbproject/private/  
.env  
*.log
```

32.4 Recuperación

Si una versión falla, se puede corregir el código y realizar un nuevo despliegue. En un entorno formal también se conserva una versión estable anterior para facilitar la reversión.

<div style="page-break-after: always;"></div>

33. Guía de uso

33.1 Acceso

1. Abrir un navegador moderno.
2. Ingresar a `https://neoportfolio-production.up.railway.app`.
3. Esperar la carga de la página inicial.

33.2 Navegación entre semanas

1. Seleccionar **Trabajos**.
2. Usar los filtros o desplazarse por la galería.
3. Elegir una tarjeta de la semana deseada.
4. Presionar **Ver proyecto**.
5. Utilizar **Volver a trabajos** para regresar.

33.3 Calculadora de ejercicios

1. Seleccionar **Ejercicios**.
2. Elegir una semana del 1 al 11.
3. Abrir un ejercicio.
4. Leer el enunciado.

5. Ingresar datos válidos.
6. Observar el resultado actualizado.
7. Consultar o copiar el código Java.

33.4 Acceso administrativo

1. Abrir la página de inicio de sesión.
2. Ingresar el usuario de demostración autorizado.
3. Escribir la contraseña configurada por el responsable.
4. Presionar **Iniciar sesión**.
5. Administrar los registros.
6. Cerrar sesión al terminar.

Referencia académica:

```
Usuario: admin_demo
Contraseña: se entrega por separado y debe cambiarse en el primer acceso.
```

<div style="page-break-after: always;"></div>

34. Mantenimiento y solución de problemas

34.1 La aplicación no conecta con MySQL

Comprobar:

- URL JDBC.
- Usuario y contraseña.
- Nombre de base de datos.
- Servicio MySQL activo.
- Variables disponibles en el servicio de aplicación.
- Compatibilidad del conector.

34.2 Error 404 al abrir una JSP

- Revisar la ruta del enlace.
- Confirmar el contexto de la aplicación.
- Usar `${pageContext.request.contextPath}`.
- Verificar el mapeo del servlet.
- No exponer vistas privadas dentro de rutas públicas.

34.3 Error con `javax` y `jakarta`

Tomcat 10 utiliza `jakarta.*`. Si el código importa `javax.servlet.*`, migrar los paquetes o usar Tomcat 9. El proyecto completo debe mantener una sola familia.

34.4 No aparecen proyectos

- Confirmar que existan registros.
- Verificar `publicado = TRUE`.
- Revisar el DAO y los nombres de atributos.
- Confirmar que JSTL esté instalada.

34.5 Los estilos no cargan

Usar una ruta con contexto:

```
<link rel="stylesheet"
      href="{pageContext.request.contextPath}/assets/css/style.css">
```

34.6 Mantenimiento periódico

- Actualizar dependencias.
- Revisar respaldos.
- Cambiar credenciales comprometidas.
- Verificar enlaces.
- Optimizar imágenes.
- Analizar registros de error.
- Probar la restauración de la base de datos.

<div style="page-break-after: always;"></div>

35. Conclusiones

NEO PORTFOLIO demuestra la integración de diseño web, programación Java, persistencia MySQL y despliegue en la nube. El resultado no se limita a una galería: incorpora organización semanal, ejercicios interactivos, visualización de código, autenticación y administración.

Logros del proyecto

- Construcción de una interfaz temática coherente.
- Aplicación del patrón MVC.
- Desarrollo de páginas JSP reutilizables.
- Persistencia mediante DAO y consultas preparadas.
- Organización de 53 ejercicios en 11 semanas.

- Creación de una galería de trabajos para 14 semanas.
- Validación de entradas y cálculo en tiempo real.
- Implementación de autenticación basada en sesiones.
- Preparación para despliegue automático.

Tecnologías aprendidas

El proyecto consolida el uso de Java, JSP, servlets, JDBC, MySQL, HTML5, CSS3, JavaScript, Apache Tomcat, Maven, GitHub y Railway. También introduce prácticas de seguridad, accesibilidad, control de versiones y documentación técnica.

Mejoras futuras

- Buscador global de proyectos y ejercicios.
- Panel con estadísticas y actividad reciente.
- Etiquetas y filtros por tecnología.
- Pruebas automatizadas más amplias.
- Migraciones con Flyway.
- Pool de conexiones.
- Recuperación segura de contraseña.
- Almacenamiento optimizado de imágenes.
- API REST para una futura aplicación móvil.
- Modo de alto contraste y selector de tema.

<div style="page-break-after: always;"></div>

36. Créditos

Desarrollo

Desarrollado por: Diego Alanya Gamarra

Periodo académico: 2do Semestre

Turno: Diurno

Año: 2025

Proyecto

Nombre: NEO PORTFOLIO

Identidad visual: Bloodpunk Cyber Demon

Tipo: Portafolio web académico y sistema interactivo de ejercicios

Java, JSP, MySQL, HTML5, CSS3, JavaScript, Apache Tomcat, Maven, GitHub y Railway.

Nota final

Este manual debe actualizarse cuando cambien la estructura de carpetas, el esquema de base de datos, las rutas, el dominio o las funciones administrativas. La documentación forma parte del producto y debe mantenerse sincronizada con el código.

<div style="page-break-after: always;"></div>

Anexo A. Dependencias Maven de referencia

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>pe.neoportfolio</groupId>
  <artifactId>neo-portfolio</artifactId>
  <version>1.0.0</version>
  <packaging>war</packaging>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>jakarta.servlet</groupId>
      <artifactId>jakarta.servlet-api</artifactId>
      <version>6.0.0</version>
      <scope>provided</scope>
    </dependency>

    <dependency>
      <groupId>org.glassfish.web</groupId>
      <artifactId>jakarta.servlet.jsp.jstl</artifactId>
      <version>3.0.1</version>
    </dependency>

    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.4.0</version>
```

```
</dependency>
</dependencies>

<build>
  <finalName>neo-portfolio</finalName>
</build>
</project>
```

Las versiones son una referencia documental. Antes de utilizarlas, se debe comprobar su compatibilidad y disponibilidad actual.

<div style="page-break-after: always;"></div>

Anexo B. Lista de comprobación para entrega

Código

- ☐ El proyecto compila.
- ☐ No existen contraseñas en el repositorio.
- ☐ Los DAO utilizan `PreparedStatement`.
- ☐ Los recursos JDBC se cierran correctamente.
- ☐ Las vistas escapan datos dinámicos.
- ☐ Las rutas administrativas están protegidas.

Contenido

- ☐ Existen 14 semanas de trabajos.
- ☐ Existen 53 ejercicios.
- ☐ Los ejercicios se agrupan en 11 semanas.
- ☐ Cada proyecto tiene título, resumen e imagen.
- ☐ Cada ejercicio tiene enunciado, entradas y código.

Interfaz

- ☐ La navegación funciona en móvil y escritorio.
- ☐ El foco de teclado es visible.
- ☐ Las imágenes incluyen texto alternativo.
- ☐ Los mensajes no dependen solo del color.
- ☐ Se respeta `prefers-reduced-motion`.

Producción

- ☐ MySQL está conectado.

- ☐ Las variables de entorno están configuradas.
 - ☐ El dominio público responde mediante HTTPS.
 - ☐ El inicio de sesión funciona.
 - ☐ Los errores no muestran información sensible.
 - ☐ Existe una copia de seguridad de la base de datos.
-

Fin del Manual de Desarrollo de NEO PORTFOLIO